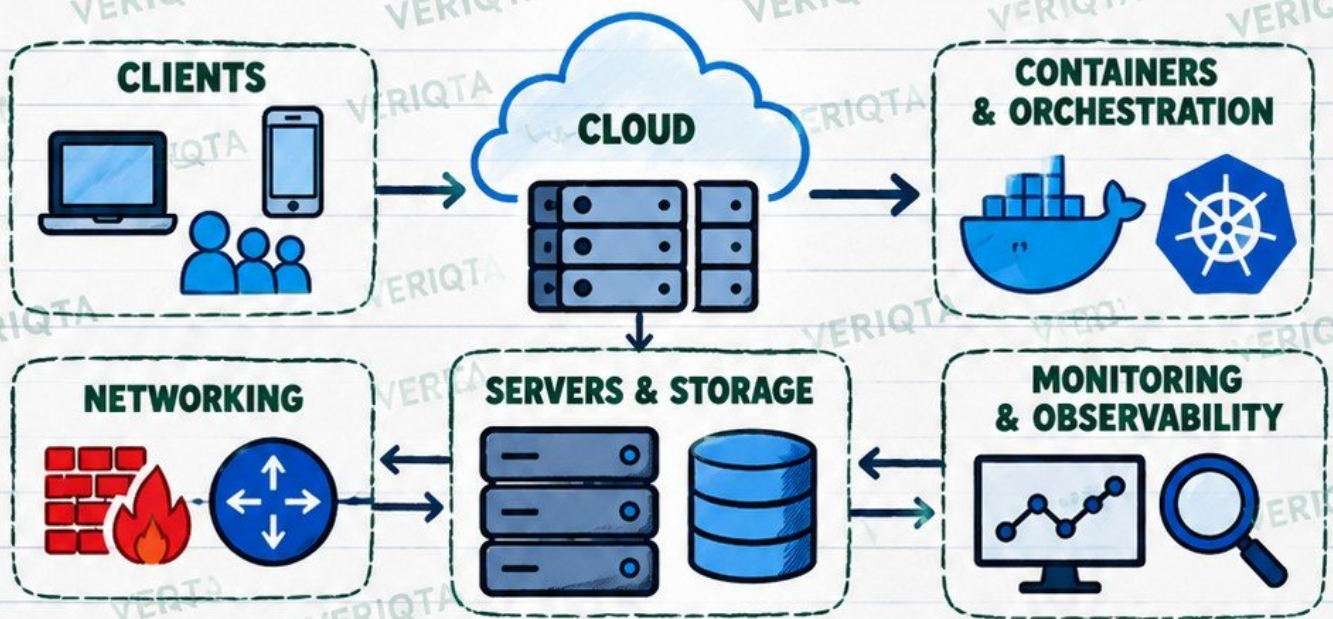


Infrastructure Engineer's Companion



- ✓ DESIGN
- ✓ BUILD
- ✓ OPERATE
- ✓ TROUBLESHOOT

- ✓ SECURE
- ✓ SCALE
- ✓ OPTIMIZE
- ✓ LEAD



@veriqta_academy



@veriqta



@veriqta



@veriqta



@veriqta

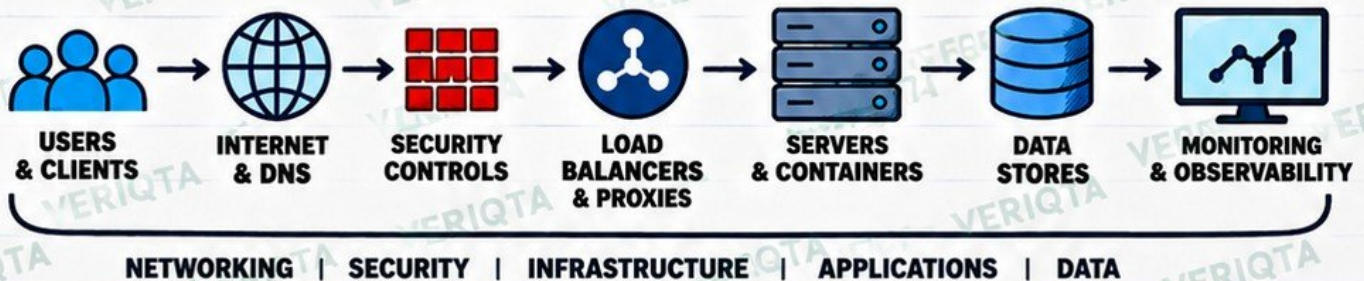


@veriqta

Senior Infrastructure Engineer **Mental Model**

Infrastructure is not a collection of tools. It is an interconnected production system that must be reliable, secure, performant, cost-efficient, operable and adaptable to change.

THE BIG PICTURE



WHAT A SENIOR ENGINEER FOCUSES ON

-  **RELIABILITY**
Keep services available and resilient.
-  **SECURITY**
Protect systems, data and access.
-  **PERFORMANCE**
Ensure low latency and efficient resource use.
-  **COST**
Deliver value while controlling spend.
-  **OPERABILITY**
Make systems easy to run, monitor and recover.
-  **CHANGE RISK**
Understand the impact of every change.

KEY MINDSET FOR SENIOR ENGINEERS

- 
 - Think in systems, not individual tools.
 - Look for root cause, not just symptoms.
 - Consider long-term impact, not just short-term fixes.
 - Balance speed with safety.
 - Design for failure and recovery.
 - Make infrastructure observable and automated.

THE SENIOR MENTAL MODEL

- 1 UNDERSTAND**
 - Know the architecture and dependencies
 - Understand normal behaviour
- 2 OBSERVE**
 - Use metrics, logs, traces and system state
 - Identify what is actually happening
- 3 CHANGE**
 - Plan and make controlled changes
 - Consider blast radius and rollback plan
- 4 VERIFY**
 - Confirm the change had the desired effect
 - Check for side effects
- 5 RECOVER**
 - If something breaks, restore service
 - Learn and prevent recurrence



@veriqta_academy



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Linux Production Triage

A rapid investigation workflow to identify and troubleshoot common Linux production issues using real evidence from the system.

TRIAGE WORKFLOW



1. UNDERSTAND

- What is the symptom?
- What changed?
- What is the impact?



2. OBSERVE

- Check system state
- Collect evidence
- Look at logs and metrics



3. INVESTIGATE

- Find the root cause
- Validate your hypothesis
- Check dependencies



4. FIX / CHANGE

- Apply the fix safely
- Consider side effects
- Make minimal changes



5. VERIFY & RECOVER

- Confirm the issue is resolved
- Monitor for recurrence

CPU & LOAD



```
uptime
top (or htop)
vmstat 1 5
```

- Check load average, CPU usage and runnable processes.
- High load can mean CPU, I/O or blocked processes.
- Use top or htop to find high CPU processes.

MEMORY



```
free -h
vmstat 1 5
cat /proc/meminfo
```

- Check used, free, cache and swap.
- Look for memory pressure or OOM activity.
- Verify if processes are consuming unusual memory.

DISK & FILESYSTEM



```
df -h
du -sh /path
lsblk
iostat -x 1 5
```

- Check disk usage and available space.
- Identify large directories and files.
- Check I/O latency and device usage.
- Look for inode exhaustion.

PROCESSES



```
ps aux --sort=-%cpu
ps aux --sort=-%mem
pstree -p
```

- Find high CPU or memory processes.
- Understand process relationships.
- Check if expected services are running.

NETWORK & CONNECTIONS



```
ss -tulnp
ss -s
lsof -i -n -P
```

- View open ports and listening services.
- Check active connections and states.
- Identify which process owns a port.

FILE DESCRIPTORS



```
ulimit -n
lsof | wc -l
lsof -p <PID>
```

- Check total open file descriptors.
- Identify processes using many files.
- High file descriptor usage can cause application failures.

I/O & SYSTEM PRESSURE



```
iostat -x 1 5
vmstat 1 5
dmesg | tail -n 50
```

- Check disk I/O, latency and throughput.
- Look for I/O wait, context switching and system pressure.
- Check kernel messages for hardware or disk errors.

LOGS & SERVICE STATUS



```
journalctl -xe
journalctl -u <service>
tail -n 100 /var/log/<app>.log
```

- Check recent system errors.
- Investigate specific service logs.
- Look for repeated errors or warnings.
- Correlate logs with the time of the issue.

QUICK HEALTH CHECK



```
systemctl status <service>
uname -a
cat /etc/os-release
```

- Verify service status.
- Check kernel and OS version.
- Confirm the system is healthy and up to date.



COMMON TRIAGE QUESTIONS

- Is it CPU, memory, disk or I/O?
- Which process or service is affected?
- What changed recently?
- Are there errors in the logs?
- Is this a single host or a wider issue?



BEGINNER MISTAKES

- Running commands without understanding the output.
- Ignoring system logs.
- Focusing on the wrong resource.
- Making changes without evidence.
- Missing the real root cause.



SENIOR ENGINEER TIP

- Always collect evidence before making changes. Use data, not assumptions.
- Look at the full system: CPU, memory, disk, network, logs and dependencies.
- Many issues are symptoms, not the root cause. Follow the evidence.



@veriqta_academy



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

CPU Saturation and Load Analysis

Understand CPU utilization, load average, runnable processes, context switching, steal time, I/O wait, throttling and CPU limits. Learn to identify real CPU exhaustion and distinguish it from other issues that look like CPU problems.

1. CPU METRICS OVERVIEW

%us	User space CPU (applications).
%sy	System space CPU (kernel).
%id	Idle CPU (available).
%wa	I/O wait (waiting for disk or network).
%st	Steal time (CPU taken by hypervisor).
%ni	Nice (lower priority processes).
%hi	Hardware interrupts.
%si	Software interrupts.

2. USEFUL COMMANDS

uptime	Show load average and uptime.
top	Real-time CPU usage by process.
htop	Interactive process viewer.
vmstat 1 5	CPU, memory, I/O and system stats.
mpstat -P ALL 1	Per-CPU utilization.
pidstat 1 5	Per-process CPU usage.
sar -u 1 5	Detailed CPU statistics.
ps aux --sort=-%cpu	Top CPU consuming processes.
lscpu	CPU architecture and cores.
cat /proc/loadavg	Raw load average values.

3. LOAD AVERAGE EXPLAINED

- Load average shows the average number of runnable and uninterruptible processes waiting for CPU over 1, 5 and 15 minutes.
- It does not show CPU percentage.
- Compare load average to the number of CPU cores.

EXAMPLE (4 CPU CORES)

Load Avg	Interpretation
0.5, 0.4, 0.3	Low load (healthy)
4.0, 3.5, 3.0	High load (saturated)
8.0, 7.5, 7.0	Severe CPU pressure

4. INTERPRETING top OUTPUT

```
top - 10:15:30 up 3 days, 2 users, load average: 4.12, 3.98, 3.50
%Cpu(s): 75.3 us, 12.1 sy, 0.0 ni, 2.4 id, 7.8 wa, 0.0 hi, 0.0 si, 2.4 st
```

High user CPU
(application workload)

High system CPU
(kernel work)

Mostly idle
(healthy)

High I/O wait
(not CPU saturation)

High steal time
(CPU taken by hypervisor)

5. COMMON CAUSES OF HIGH CPU

- A process using too much CPU.
- Too many runnable processes.
- Inefficient code or infinite loops.
- High traffic or increased workload.
- Background jobs (backup, scans, indexing).
- Container CPU limits causing throttling.
- VM steal time (noisy neighbours).
- Too many interrupts or context switching.
- Misconfigured autoscaling.
- Malicious activity (crypto mining).

6. DISTINGUISH REAL CPU SATURATION

Scenario	Symptoms	How to Confirm	Key Difference
CPU Exhaustion	High %us + high load run queue full	top, vmstat, mpstat	CPU cores fully busy
I/O Wait (looks like CPU)	High %wa, high load	iostat, vmstat, top	Processes waiting on disk or network
Steal Time (looks like CPU)	High %st, high load	top, mpstat	Hypervisor taking CPU
Misleading Load Average	High load but low CPU%	uptime + top	Many blocked processes (not CPU bound)
CPU Throttling (container)	High load, limited CPU% processes slow	cat /sys/fs/cgroup/cpu/... pidstat, top	Cgroup CPU limit reached
High Interrupts	High %si or %hi	top, sar -l, dmesg	Hardware or network issue

COMMON MISTAKES

- Assuming high load average always means CPU saturation.
- Ignoring I/O wait and blaming the application.
- Not checking steal time on cloud VMs.
- Focusing on the wrong process.
- Ignoring container CPU limits and throttling.
- Not looking at system logs and metrics.

SENIOR ENGINEER TIPS

- Always check the full CPU breakdown, not just total %.
- Compare load average with the number of CPU cores.
- Use multiple tools to confirm your findings.
- Look at I/O, network and memory before assuming CPU.
- Investigate container and VM limits.
- Correlate with application logs and metrics.
- Identify what changed before the issue started.

REMEMBER THIS

- High CPU is a symptom, not a cause.
- Load average shows demand, not usage.
- High I/O wait is a storage or network issue.
- High steal time is a virtualization issue.
- Always verify the root cause with evidence.
- Fixing the right problem prevents recurrence.



@veriqta_academy



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Memory Pressure and OOM Investigation

Understand how Linux uses memory, what causes memory pressure, and how to investigate OOM (Out of Memory) issues. Learn to find the affected process, check logs, review limits and identify the real root cause.

1. LINUX MEMORY COMPONENTS

RAM	Physical memory used by the OS and applications.
Cache	Used to speed up disk access. Reclaimed when needed.
Swap	Disk space used when RAM is low.
Resident Memory (RSS)	Actual physical memory used by a process.
Virtual Memory	Total address space used by a process.

2. USEFUL COMMANDS

free -h	View RAM, cache and swap usage.
top	Real-time process memory usage.
htop	Interactive process viewer.
vmstat 1 5	Memory, CPU and system stats.
ps aux --sort=-%mem	Top memory consuming processes.
smem -r head	Detailed memory usage by process.
cat /proc/meminfo	Detailed memory information.
cat /proc/<pid>/status	Memory details for a process.
dmesg grep -i oom	Check OOM killer events.
journalctl -k grep -i oom	View kernel OOM logs.

3. READ free -h OUTPUT

\$ free -h						
	total	used	free	shared	buff/cache	available
Mem:	15Gi	13Gi	500Mi	300Mi	1.5Gi	1.2Gi
Swap:	4Gi	2.1Gi	1.9Gi			

High used memory

Very low free memory

Cache can be reclaimed

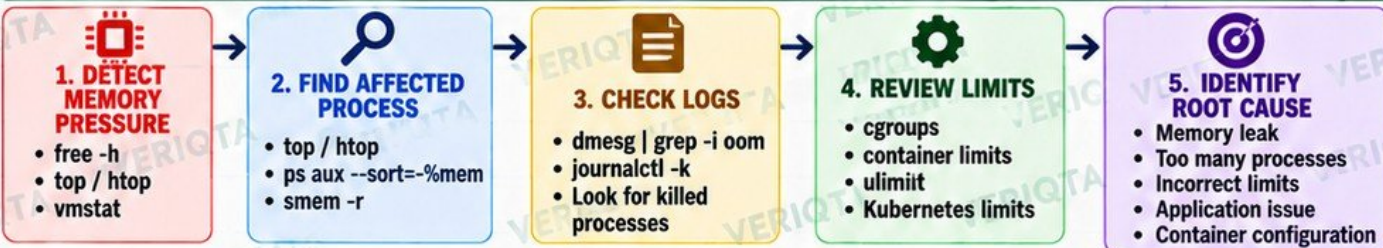
Low available memory = pressure

4. UNDERSTANDING OOM KILLER



- The Linux OOM (Out of Memory) killer terminates processes when memory is critically low.
- It chooses a process based on memory usage and the OOM score.
- You will see entries in dmesg or journalctl with **"Out of memory: Killed process"**.

5. INVESTIGATION FLOW



6. COMMON CAUSES OF MEMORY PRESSURE

- Memory leak in application.
- Too many processes or high concurrency.
- Large cache or buffers.
- Insufficient container or VM memory limits.
- High memory usage from background jobs (backup, indexing).
- Swap usage due to low RAM.
- Misconfigured applications or services.
- Kernel memory usage or filesystem cache growth.

7. CHECK CONTAINER MEMORY LIMITS

- docker stats**
- docker inspect <container>**
- Look for memory limit.
- kubectl describe pod <pod>**
- Check **resources.limits.memory**
- Check **resources.requests.memory**
- Look for **OOMKilled** in events.

8. KEY INDICATORS

- High % memory in top/htop.
- Swap usage increasing.
- OOM killer logs in dmesg.
- Processes with high RSS.
- Container or cgroup limits reached.
- Application crashes during high traffic/load.
- System becomes slow or unresponsive.

9. REMEMBER THIS

- High memory usage is a symptom, not always the root cause.
- Always check logs to confirm OOM killer activity.
- Look at the full system: processes, containers, limits, logs and workload.
- Fix the root cause, not just increase memory.
- Monitor memory usage to prevent recurrence.

10. REAL WORLD EXAMPLE (OOM LOG)

```
Out of memory: Killed process 12345 (java) total-vm:4097152kB,
anon-rss:307200kB, file-rss:0kB, shmem-rss:0kB, UID:1000
oom_score_adj:0
```



@veriqta_academy



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Disk, Filesystem and I/O Failures

Learn how to investigate disk space issues, inode exhaustion, filesystem problems, I/O latency, IOPS, throughput, blocked processes and deleted-but-open files. Understand the real cause when disk shows 100% and how to fix storage-related failures.

1. KEY CONCEPTS

Disk Capacity	Total storage space available.
Inodes	Track files and directories. Can be exhausted even if disk space is free.
Filesystem	How data is stored (ext4, xfs, etc).
I/O Latency	Time to read or write data.
IOPS	Number of read/write operations per second.
Throughput	Amount of data transferred per second.
Blocked Processes	Processes waiting for disk I/O.
Deleted-but-Open Files	Files deleted but still held open by a process.
Storage Saturation	Disk or I/O too busy, causing slow apps.

2. USEFUL COMMANDS

Command	Purpose
<code>df -h</code>	Check disk space usage.
<code>df -hi</code>	Check inode usage.
<code>du -sh /path</code>	Find large directories.
<code>du -ah /path sort -rh head</code>	Find largest files and directories.
<code>lsblk</code>	Show block devices and mounts.
<code>mount column -t</code>	Check mounted filesystems.
<code>iostat -x 1 5</code>	Check I/O usage, latency and throughput.
<code>vmstat 1 5</code>	System-wide I/O and process state.
<code>lsof +L1</code>	Find deleted but open files.
<code>ps aux --sort=-%iowait</code>	Find processes waiting on disk I/O.
<code>dmesg grep -i error</code>	Check disk or filesystem errors.
<code>smartctl -a /dev/sdX</code>	Check disk health (if available).

3. CHECK DISK SPACE AND INODES

Disk space usage



`df -h`

- Shows total, used, free space.
- Identify which filesystem is full.

Inode usage



`df -hi`

- Shows inode usage.
- If inodes are 100%, you cannot create new files even if space is free.

4. FIND LARGE FILES AND DIRECTORIES

Find large directories



`du -sh /path`

- Check which directory is using the most space.
- Start from root (/) or application directories.

Find largest files



`du -ah /path | sort -rh | head`

- Show largest files first.
- Helps find log files, dumps or app-generated large files.

5. CHECK I/O PERFORMANCE



`iostat -x 1 5`

- Check I/O utilization, latency, IOPS.
- Look at %util (high = saturated).
- Check await (high = slow I/O).

Key iostat fields

%util	Disk busy time (close to 100% = saturation)
await	Average I/O wait time (high = slow)
r/s w/s	Read/write operations per second (IOPS)
rkB/s kB/s	Read/write throughput

6. DETECT DELETED-BUT-OPEN FILES



`lsof +L1`

- Shows files that have been deleted but are still open by a running process.
- Common with log files.

7. FIND PROCESSES BLOCKED ON I/O



`ps aux --sort=-%iowait`

- Show processes with highest I/O wait.
- Helps identify which process is causing disk slowness.

8. CHECK FILESYSTEM AND DISK HEALTH



`dmesg | grep -i error`
`smartctl -a /dev/sdX`

- Look for filesystem or disk errors.
- Check for read/write failures.
- Verify disk health (SMART).

9. CLASSIC INVESTIGATION FLOW

- 1 Check disk space and inodes (`df -h`, `df -hi`)
- 2 Find large files/directories (`du`)
- 3 Check deleted-but-open files (`lsof +L1`)
- 4 Check I/O performance (`iostat`, `vmstat`)
- 5 Identify blocking processes (`ps`, `top`)
- 6 Review logs for errors (`dmesg`, `journalctl`)
- 7 Confirm root cause and apply the fix

10. COMMON CAUSES

- Application logs growing too large.
- Leftover core dumps or backup files.
- Inode exhaustion (too many files).
- Deleted files still open by a process.
- High I/O from database, antivirus or backup jobs.
- Container or VM disk limits reached.

11. KEY TAKEAWAYS

- Disk full can be space or inode related.
- Deleted files can still consume space.
- High I/O can make the system slow without showing 100% disk usage.
- Always check metrics, logs and processes.
- Fix the root cause, not just free space.



@veriqta_academy



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Production Networking Investigation

Trace a connection from your client to the application using real network components. Use diagnostic tools to find where the connection is failing and why. Learn to distinguish DNS failure, routing issues, firewall blocks, load balancer problems, host/port issues and TLS failures.

1. THE CONNECTION PATH



2. USEFUL COMMANDS

Command	Purpose
<code>dig example.com</code>	Check DNS resolution and returned IP.
<code>dig example.com +trace</code>	Trace full DNS path (advanced).
<code>ip a ip r</code>	Check network interfaces and routing.
<code>tracert example.com</code>	See the network path to the destination.
<code>ss -tulnp</code>	Check listening ports and connections.
<code>curl -v https://example.com</code>	Test HTTP/HTTPS and view handshake.
<code>tcpdump -i eth0 host <ip></code>	Capture and inspect network traffic.
<code>telnet <ip> <port></code>	Quick test if a port is reachable.
<code>nc -zv <ip> <port></code>	Check port connectivity (no data sent).

4. COMMON FAILURES AND HOW TO IDENTIFY THEM

Failure Type	Typical Symptoms	What to Check	Example / Tool
DNS Failure	- Cannot resolve hostname - Temporary failure in name resolution	DNS records, resolver, network	<code>dig example.com</code> <code>dig +trace</code>
Connection Timeout	- No response - Hangs until timeout	Network path, firewall, routing	<code>tracert</code> <code>curl -v</code>
Connection Refused	- Immediate refusal - Port not open	Service not running, wrong port, firewall rejecting	<code>ss -tulnp</code> <code>nc -zv <ip> <port></code>
TLS Failure	- SSL/TLS handshake error - Certificate issues	Certificate, hostname, TLS configuration	<code>curl -v https://...</code> <code>openssl s_client</code>
Slow Response	- High latency - Intermittent timeouts	Network, load balancer, backend performance	<code>tracert</code> <code>tcpdump</code> , <code>curl -v</code>

5. REAL EXAMPLE: TRACE A PROBLEM

```

$ dig app.example.com
;; ANSWER SECTION:
app.example.com.      300 IN A 203.0.113.10

$ traceroute 203.0.113.10
 1 10.0.0.1
 2 192.168.1.1
 3 * * * (timeout)
  
```

Stops at hop 3 →
network or firewall issue

3. INVESTIGATION FLOW

- Check DNS**
 - `dig example.com`
 - Confirm it resolves to the correct IP.
- Check Routing**
 - `tracert example.com`
 - Check for network delays or drops.
- Check Firewall / Security**
 - Verify security groups, ACLs, firewalls.
 - Confirm the port is allowed.
- Check Load Balancer**
 - Verify the load balancer is healthy.
 - Check target group health and routing.
- Check Host and Port**
 - `ss -tulnp` on the server.
 - Confirm the application is listening.
- Test the Application**
 - `curl -v http://<ip>:<port>`
 - Check response, errors or timeouts.

6. SENIOR ENGINEER TIPS

- Always test each layer in order.
- Use multiple tools to confirm findings.
- Compare from different networks.
- Look at both client and server logs.
- Check for asymmetric routing.
- Verify security rules and NAT.
- Don't just test the port, test the app.
- Capture evidence with `tcpdump` when needed.

7. REMEMBER THIS

- DNS issues fail early.
- Timeouts usually mean a network path or firewall problem.
- Connection refused means the port is closed or no service is listening.
- TLS errors are not network issues, they are certificate or configuration issues.
- A successful connection to a port does not mean the application is healthy.



@veriqta_academy



@veriqta



@veriqta



@veriqta



@veriqta

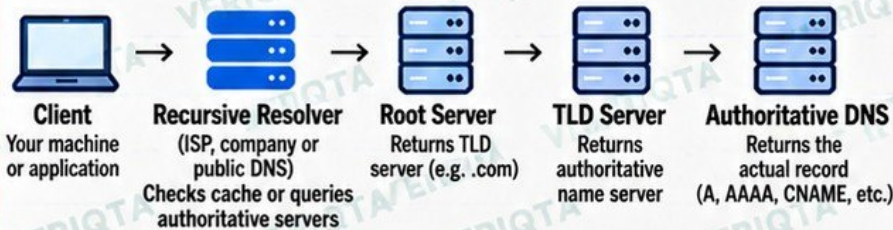


@veriqta

DNS Failure Analysis

Understand how DNS works, common DNS failure scenarios and how to troubleshoot them. Learn about recursive resolution, authoritative DNS, caching, TTLs, record types, propagation, resolver failures, split DNS and Kubernetes/cloud DNS.

1. HOW DNS WORKS (HIGH LEVEL)



2. DNS RECORD TYPES

Type	Purpose
A	IPv4 address
AAAA	IPv6 address
CNAME	Alias to another domain
MX	Mail server for the domain
NS	Authoritative name server
TXT	Text record (SPF, DKIM, etc.)
SRV	Service location (rare)
PTR	Reverse DNS lookup
SOA	Zone information (serial, refresh, etc.)

3. IMPORTANT DNS CONCEPTS

Recursive Resolution	Client asks a resolver to find the answer.
Authoritative DNS	Holds the actual zone and records.
Caching	Resolvers cache responses to improve speed.
TTL (Time To Live)	How long a record is cached (in seconds).
Propagation	Time for DNS changes to be visible globally.
Resolver Failures	DNS server unreachable or misconfigured.
Split DNS	Different answers for internal vs external users.
Kubernetes DNS	CoreDNS resolves service names inside the cluster.
Cloud DNS	Cloud provider DNS (Route 53, Cloud DNS, Azure DNS, etc.).

4. USEFUL COMMANDS

Command	Purpose
<code>dig domain.com</code>	Detailed DNS query.
<code>dig domain.com A</code>	Query specific record type.
<code>dig +trace domain.com</code>	Trace full DNS resolution path.
<code>nslookup domain.com</code>	Basic DNS lookup.
<code>host domain.com</code>	Simple DNS query.
<code>resolvectl status</code>	Check system DNS configuration.
<code>cat /etc/resolv.conf</code>	View configured DNS servers.
<code>dig @8.8.8.8 domain.com</code>	Query a specific DNS server.
<code>dig -x <IP></code>	Reverse DNS lookup (PTR).
<code>kubectl exec -it <pod> -- nslookup svc</code>	DNS test inside Kubernetes.
<code>tcpdump -n port 53</code>	Capture DNS traffic.

5. COMMON DNS FAILURE SCENARIOS

Failure Type	Symptoms	Possible Cause	What to Check
DNS Not Resolving	• Cannot resolve hostname • NXDOMAIN	Wrong domain, missing record	• <code>dig</code> , <code>nslookup</code>
Slow DNS Resolution	• High latency • Intermittent timeouts	Resolver issues, network, large response	<code>dig +trace</code> , <code>tcpdump</code>
Wrong IP Address	• Resolves to wrong IP	DNS cache, old record, split DNS	• Check TTL, clear cache
Intermittent Failures	• Sometimes works • Sometimes fails	Multiple resolvers, network, propagation	• Test from different networks
Internal vs External	• Different IPs returned	Split DNS configuration	• Test from internal and external
Kubernetes DNS Issues	• Pod cannot resolve service	CoreDNS issue, wrong service name	<code>kubectl logs</code> , <code>nslookup</code>
Cloud DNS Issues	• Works locally, not in cloud	VPC DNS, security group, private zone	• Check VPC DNS settings

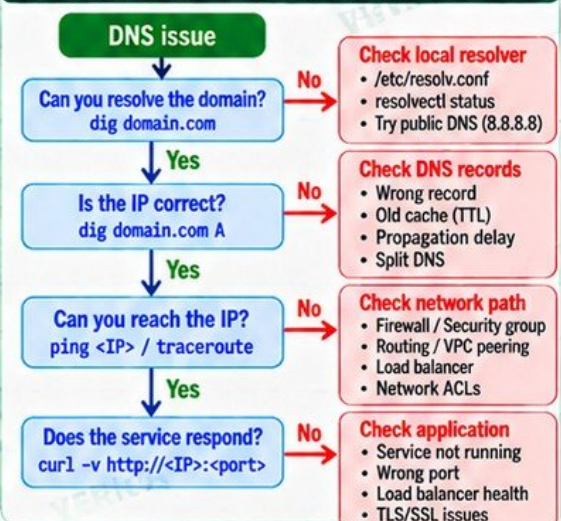
7. CHECK DNS PROPAGATION AND CACHE

- DNS changes can take time to propagate.
- Check TTL value: `dig domain.com SOA`
- Test from multiple DNS servers and locations.
- Clear local cache if needed:
 - `systemd-resolve --flush-caches`
 - `sudo nsdc -i hosts`
 - `sudo rndc flush` (on DNS server)

8. KEY TAKEAWAYS

- DNS issues are often not application issues.
- Always test each layer (resolver, network, authoritative).
- Compare results from multiple locations.
- Understand TTL and caching behavior.
- Use the right tools to get evidence.
- Check for split DNS and cloud-specific configuration.

6. DNS TROUBLESHOOTING DECISION PATH



9. REMEMBER THIS

- DNS must work before anything else.
- Correct DNS record does not guarantee the service is reachable.
- Propagation delay is normal after changes.
- Always confirm the IP and test the network path.
- In Kubernetes, use service names, not external domains.
- Collect evidence (dig, traces, logs) before making changes.



@veriqta_academy



@veriqta



@veriqta



@veriqta



@veriqta

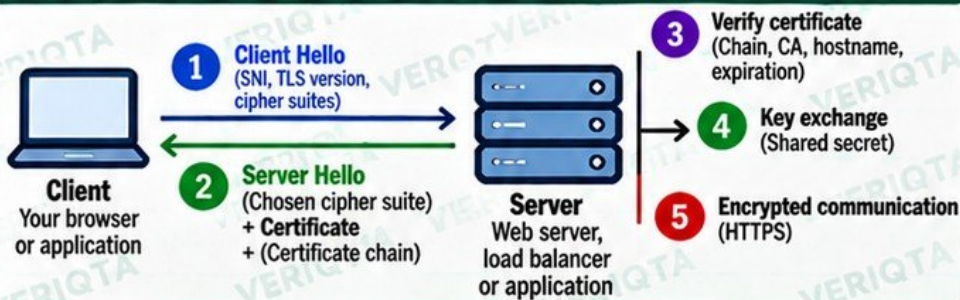


@veriqta

TLS, Certificates and Secure Traffic

Understand how TLS works, the certificate chain, certificate authorities, hostname validation, expiration, SNI and common certificate failures. Learn how certificates are used with ingress controllers, reverse proxies, load balancers and public services.

1. TLS HANDSHAKE (SIMPLIFIED FLOW)



2. CERTIFICATE CHAIN



3. KEY TLS CONCEPTS

TLS	Encrypts traffic between client and server.
Certificate	Proves the identity of the server.
Certificate Chain	Server cert + intermediate CA(s) + root CA.
Certificate Authority (CA)	Issues and signs certificates.
Hostname Validation	Client checks if the certificate matches the domain.
Expiration	Certificate has a valid date range.
SNI (Server Name Indication)	Client tells the server which domain it's requesting (multiple domains on one IP).
Private Key	Kept secret on the server, used to decrypt data.

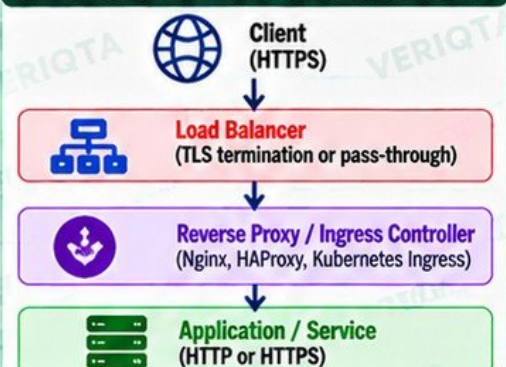
4. USEFUL COMMANDS

Command	Purpose
<code>openssl s_client -connect example.com:443 -servername example.com</code>	View certificate details, chain and handshake.
<code>openssl x509 -in cert.pem -noout -text</code>	Inspect certificate information.
<code>openssl x509 -in cert.pem -noout -dates</code>	Check certificate expiration date.
<code>curl -v https://example.com</code>	View TLS handshake and certificate validation.
<code>echo openssl s_client -connect example.com:443</code>	Quick certificate check.
<code>openssl verify -CAfile ca.pem cert.pem</code>	Verify certificate chain.

5. COMMON CERTIFICATE FAILURES

Failure Type	Error Message (Example)	Cause	How to Fix
Expired Certificate	• certificate has expired	Certificate past valid date.	Renew certificate.
Hostname Mismatch	• hostname mismatch	Cert not valid for the domain.	Use correct certificate with proper SAN/CN.
Untrusted Certificate	• unable to get local issuer certificate	CA not trusted.	Install proper CA or use public CA.
Incomplete Chain	• incomplete certificate chain	Missing intermediate certificates.	Configure full chain on server.
Wrong Certificate	• wrong version number	Non-TLS service or wrong port.	Use correct port (443) and valid cert.
SNI Misconfiguration	• default certificate shown	Server not using SNI correctly.	Enable SNI and configure domain certificates.

6. CERTIFICATES IN REAL ARCHITECTURE



7. CHECK CERTIFICATE DETAILS (EXAMPLE)

```
$ openssl s_client -connect example.com:443 -servername example.com
...
subject=CN = example.com
issuer=CN = R3, O = Let's Encrypt, C = US
Not Before: Aug 10 00:00:00 2024 GMT
Not After : Nov 08 23:59:59 2024 GMT
...
Verify return code: 0 (ok)
```

8. KEY TAKEAWAYS

- Always use HTTPS for public services.
- Check certificate expiration regularly.
- Use trusted CAs (e.g. Let's Encrypt).
- Ensure hostname matches the certificate.
- Configure the full certificate chain.
- Use SNI for multiple domains on one IP.
- Monitor certificate expiry and automate renewal.

9. REMEMBER THIS

- TLS Encrypts the data, certificates prove the identity.
- Most TLS failures are certificate, hostname or chain issues.
- Use the right certificate on the right server and port.
- In Kubernetes, check the Ingress/Cert and secret configuration.
- Always test from the client side (using openssl or curl).



@veriqta_academy



@veriqta



@veriqta



@veriqta



@veriqta

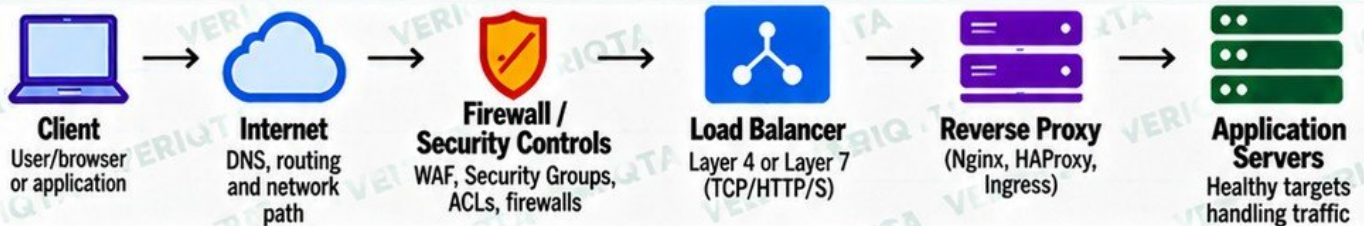


@veriqta

Load Balancers, Proxies and Traffic Flow

Understand how load balancers and proxies work, Layer 4 vs Layer 7, health checks, connection draining, session persistence, upstream failures and how to trace a request from the client to the application in a real production environment.

1. TYPICAL PRODUCTION TRAFFIC FLOW



2. LAYER 4 vs LAYER 7 LOAD BALANCING

Feature	Layer 4 (L4)	Layer 7 (L7)
Works at	Transport layer (TCP/UDP)	Application layer (HTTP/HTTPS)
Uses	IP address and port	HTTP headers, host, path, cookies
Common tools	AWS NLB, F5 (L4), HAProxy (TCP)	AWS ALB, Nginx, HAProxy (HTTP)
Routing	Based on IP and port	Based on host, path, header, cookie
Use cases	High performance, low latency, non-HTTP traffic	Web applications, multi-domain, advanced routing
Examples	TCP 80/443, database traffic	Host-based routing, path-based routing, canary releases

3. KEY CONCEPTS

Reverse Proxy	Receives client requests and forwards to backend servers.
Health Check	Checks if targets are healthy.
Connection Draining	Allows existing connections to finish before removing a target.
Session Persistence	Keeps a user on the same backend (server) using cookies or IP hash.
Upstream Failure	Backend server or service is down or slow.
Unhealthy Target	Target fails health checks and is removed from rotation.
Ingress Controller	Kubernetes component that manages external access to services.

4. USEFUL COMMANDS

Command	Purpose
<code>curl -v https://example.com</code>	See request/response details and headers.
<code>curl -I https://example.com</code>	Check HTTP headers only.
<code>dig example.com</code>	Check DNS resolution.
<code>nslookup example.com</code>	DNS lookup (basic).
<code>openssl s_client -connect example.com:443</code>	Test TLS and certificate.
<code>ss -tulnp</code>	Check listening ports on the server.
<code>sudo tcpdump -i eth0 port 80 or 443</code>	Capture network traffic.
<code>curl -H "Host: example.com" http://<IP></code>	Test host-based routing.

5. LOAD BALANCER HEALTH CHECKS



6. COMMON ISSUES AND SYMPTOMS

Issue	Symptoms	Possible Cause	How to Check
502 Bad Gateway	Gateway error	Upstream server down	Check target health checks, backend logs
503 Service Unavailable	Service unavailable	No healthy targets	Check load balancer target health and auto-scaling
504 Gateway Timeout	Request timeout	Slow backend or network issue	Check backend response time, network latency
Connection refused	Cannot connect	Service not listening or firewall	<code>ss -tulnp</code> , firewall rules
SSL/TLS error	Certificate or handshake error	Expired/invalid cert, wrong SNI	<code>openssl s_client</code> , check cert
Inconsistent routing	Users hitting different servers	No session persistence	Check LB config (cookie/IP hash)

8. EXAMPLE: CHECK HTTP RESPONSE AND HEADERS

```

$ curl -v https://example.com
> GET / HTTP/1.1
> Host: example.com
> User-Agent: curl/8.0
>
< HTTP/1.1 200 OK
< Server: nginx
< Content-Type: text/html
  
```

Request sent to load balancer/proxy

Response from application

9. KEY TAKEAWAYS

- Understand the full traffic flow from client to application.
- Know the difference between L4 and L7 load balancing.
- Use health checks and connection draining.
- Investigate from DNS → network → LB → proxy → backend.
- Check logs at each layer.
- Confirm root cause before making changes.

7. TRACE A PRODUCTION REQUEST

- Resolve DNS
`dig example.com`
- Check network path
`traceroute example.com`
- Test TLS and certificate
`openssl s_client -connect example.com:443`
- Send HTTP request
`curl -v https://example.com`
- Check load balancer
Review target health and access logs
- Verify backend server
Check application logs and response time

10. REMEMBER THIS

- A 200 OK from the load balancer does not mean the application is healthy.
- Always check target health and backend logs.
- TLS and certificate issues can look like network or application problems.
- Use multiple tools to confirm where the request is failing.
- In Kubernetes, check the Ingress, Service and Endpoints when traffic is not reaching pods.

Infrastructure as Code at Production Scale

Learn how to use Terraform (or equivalent IaC) safely at scale. Understand state, modules, dependency graphs, remote state, locking, drift, plan review, immutable changes, imports and best practices. Learn why terraform apply should never become a reflex.

1. IaC WORKFLOW WITH TERRAFORM



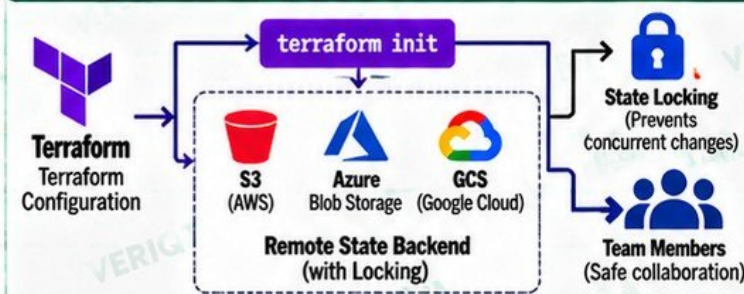
2. KEY IaC CONCEPTS

Concept	What it Means
Configuration	Declarative code that defines infrastructure.
State	Tracks real infrastructure resources and their settings.
Modules	Reusable, versioned building blocks.
Dependency Graph	Terraform understands relationships and order.
Remote State	Stores state in a remote backend (S3, Azure, GCS, etc.).
State Locking	Prevents multiple people from making changes at once.
Drift	Difference between code and actual infrastructure.
Plan Review	Shows exactly what will change before applying.
Immutable Changes	Replace instead of in-place updates for safer changes.
Imports	Bring existing resources under Terraform management.

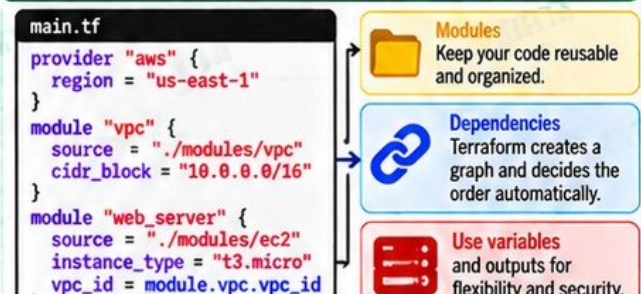
3. COMMON TERRAFORM COMMANDS

Command	Purpose
<code>terraform init</code>	Initialize the working directory.
<code>terraform validate</code>	Check configuration for errors.
<code>terraform fmt</code>	Format code for consistency.
<code>terraform plan</code>	Show what changes will be made.
<code>terraform apply</code>	Create or update infrastructure.
<code>terraform destroy</code>	Remove infrastructure (use with caution).
<code>terraform state list</code>	List resources in the state.
<code>terraform state show <resource></code>	Show details of a resource.
<code>terraform import <resource> <id></code>	Import existing resource.
<code>terraform taint <resource></code>	Mark resource to be recreated.

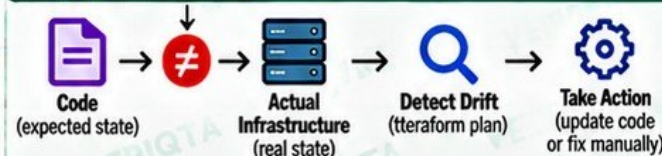
4. REMOTE STATE AND LOCKING



5. EXAMPLE: SIMPLE TERRAFORM CONFIGURATION



6. DRIFT DETECTION



7. WHY terraform apply SHOULD NOT BE A REFLEX



8. SAFE INFRASTRUCTURE MODIFICATION PROCESS



9. KEY TAKEAWAYS

- ✓ Treat infrastructure as code, not clicks.
- ✓ Use remote state and locking.
- ✓ Always review the plan before apply.
- ✓ Keep modules small and reusable.
- ✓ Detect and fix drift regularly.
- ✓ Prefer immutable changes.
- ✓ Have a rollback and recovery plan.

10. REMEMBER THIS

- Code can be versioned. Manual console changes cannot.
- terraform apply is powerful and can be dangerous.
- Plan, review, and understand before applying.
- Safety, reproducibility and discipline keep your systems healthy.



@veriqta_academy



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

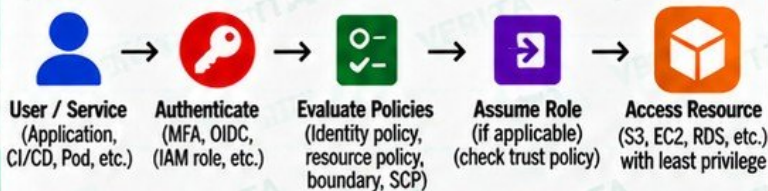
Cloud Identity and Least Privilege

Understand cloud identity, roles, service identities, policies, temporary credentials, trust relationships, permission boundaries, workload identity and privilege escalation risks. Learn how to investigate an AccessDenied failure without granting excessive permissions.

1. CLOUD IDENTITY COMPONENTS



2. IDENTITY AND ACCESS FLOW (EXAMPLE)



4. KEY CONCEPTS

Concept	Purpose
Least Privilege	Grant only the permissions required.
Policy	Defines allowed or denied actions.
Trust Relationship	Determines who can assume a role.
Temporary Credentials	Short-lived, more secure access (STS, OIDC).
Permission Boundary	Sets the maximum permissions for a principal.
Workload Identity	Identity for containers and workloads (IRSA, Workload Identity).
Privilege Escalation	Gaining more permissions than intended (e.g., pass role, create policy).
Resource Policy	Policy attached to a resource (e.g., S3 bucket policy).
SCP	Organization-level guardrails (restrict across accounts).
Session Policies	Further restrict permissions on assumed roles.

6. USEFUL CLI COMMANDS (AWS EXAMPLES)

Command	Purpose
<code>aws sts get-caller-identity</code>	See current identity (user/role/account).
<code>aws iam get-user</code>	Get details about the current IAM user.
<code>aws iam list-attached-user-policies --user-name <user></code>	Check attached policies for a user.
<code>aws iam list-attached-role-policies --role-name <role></code>	Check attached policies for a role.
<code>aws iam get-role --role-name <role></code>	View role details and trust policy.
<code>aws iam simulate-principal-policy --policy-source-arn <arn> --action-names <action></code>	Test if an action is allowed.
<code>aws s3api get-bucket-policy --bucket <bucket></code>	Check S3 bucket resource policy.
<code>aws organizations describe-effective-policy --policy-type SERVICE_CONTROL_POLICY</code>	Check SCP affecting an account.
<code>aws configure list</code>	See which credentials are being used.

8. COMMON PRIVILEGE ESCALATION RISKS



- Overly permissive policies (e.g., *, AdministratorAccess).
- Ability to pass roles (iam:PassRole).
- Modifying trust relationships.
- Creating or attaching policies.
- Exploiting service misconfigurations (e.g., S3, Lambda, EKS, ECS).
- Compromised credentials or long-lived access keys.
- Missing permission boundaries or SCP.

9. KEY TAKEAWAYS



- Always use least privilege.
- Understand the full policy evaluation chain (identity policy, resource policy, boundary, SCP).
- Use temporary credentials where possible.
- Review and test changes before production.
- Never grant excessive permissions to fix an error.
- Monitor and audit identity and access.
- Use conditions and boundaries for additional security.

3. COMMON IDENTITY TYPES

Identity Type	Use Case
User	Human access (console/CLI)
Role	Temporary access for services or users
Service Account	Kubernetes workload identity
IAM Role for Service	EC2, Lambda, ECS, etc.
Federated Identity	OIDC (GitHub Actions, CI/CD, SSO)
Service Identity	Application or managed service
Cross-Account Role	Access resources in another account
Permission Boundary	Limit maximum permissions
SCP (Org Level)	Restrict permissions across accounts

5. INVESTIGATING AN AccessDenied FAILURE

- 1 Identify the Error**
 - Read the full error message.
 - Identify the action, resource and principal.
- 2 Identify the Identity**
 - Who is making the request? (user, role, service)
 - Check if temporary credentials are used.
- 3 Check Identity Policies**
 - Verify attached policies and permissions.
 - Look for explicit Deny.
- 4 Check Resource Policies**
 - S3, KMS, SNS, etc. may have their own policies.
 - Ensure the principal is allowed.
- 5 Check Permission Boundaries / SCP**
 - Confirm no boundary or org policy is blocking.
 - Look for Deny in SCP.
- 6 Verify Trust Relationships**
 - If assuming a role, check the trust policy.
 - Ensure the identity is allowed to assume it.
- 7 Check Conditions**
 - Look for conditions (IP, VPC, MFA, tags, etc.).
 - Confirm all conditions are met.
- 8 Fix with Least Privilege**
 - Grant the minimum required permission.
 - Avoid using broad permissions like * or admin.

7. EXAMPLE: AccessDenied TROUBLESHOOTING

```
$ aws s3 ls s3://prod-logs
An error occurred (AccessDenied) when calling the ListObjectsV2 operation: Access Denied
```

- 1 Check identity**

```
aws sts get-caller-identity
```
- 2 Check identity policy**

```
aws iam list-attached-role-policies --role-name <role>
```
- 3 Check bucket policy**

```
aws s3api get-bucket-policy --bucket prod-logs
```
- 4 Check SCP / boundary**

```
aws organizations describe-effective-policy --policy-type SERVICE_CONTROL_POLICY
```
- 5 Fix with least privilege**

```
Add only the required s3:ListBucket permission for the specific bucket.
```

10. REMEMBER THIS

AccessDenied is a security control, not just an error.

- Understand the identity, action, resource and policy evaluation flow.
- Prioritize identifying and fixing work by granting broad permissions.
- Always review changes and monitor



@veriqta_academy



@veriqta



@veriqta



@veriqta

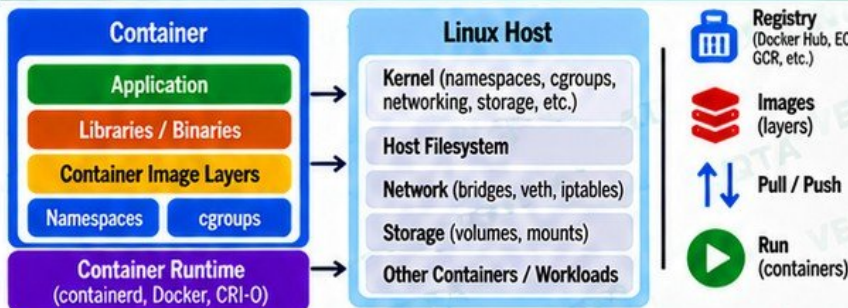


@veriqta

Containers Beyond the Basics

Understand how containers work under the hood: namespaces, cgroups, images, layers, runtime, networking, volumes, resource limits, signals, PID 1, container exits and host dependencies.
Learn to connect container symptoms back to the underlying Linux host.

1. CONTAINER ARCHITECTURE (BIG PICTURE)



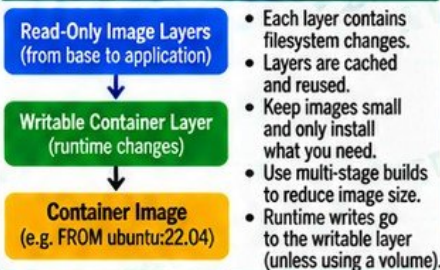
2. NAMESPACES (ISOLATION)

Namespace	Isolates
PID	Process ID space (containers see their own PIDs)
NET	Network interfaces, routing, ports
MNT	Filesystem mounts
UTS	Hostname and domain name
IPC	Inter-process communication (shared memory, message queues)
USER	User and group IDs (root inside ≠ root outside)
CGROUP	Resource limits (CPU, memory, I/O, etc.)

3. cgroups (RESOURCE CONTROL)

Resource	What it Controls
CPU	CPU shares, limits, quotas
Memory	Memory limit, swap limit, OOM behavior
I/O	Disk I/O weight and throttling
PIDs	Maximum number of processes
Devices	Access to host devices
Network	Network bandwidth (via CNI and plugins)

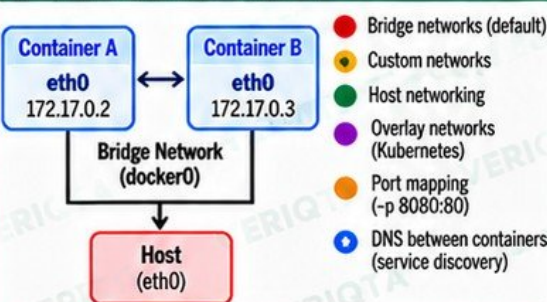
4. CONTAINER IMAGE LAYERS



5. CONTAINER RUNTIME

Docker	Popular runtime and tooling (docker run, docker build, etc.).
containerd	Industry standard runtime (used by Kubernetes).
CRI-O	Kubernetes-native runtime.
runc	Low-level OCI runtime.
Podman	Daemonless container runtime (alternative to Docker).

6. CONTAINER NETWORKING



7. VOLUMES AND STORAGE

Type	Purpose
Named Volume	Persistent data managed by Docker/Kubernetes.
Bind Mount	Mount host directory into container.
tmpfs	In-memory storage (non-persistent).
Read-Only Mount	Prevent container from modifying files.
Storage Driver	Manages container filesystem layers (overlay2, etc.).

8. RESOURCE LIMITS (EXAMPLE)

<pre># Limit CPU and memory docker run -d --name web \ --cpus="1.0" \ --memory="512m" \ --memory-swap="1g" nginx:latest</pre>	
Option	Meaning
--cpus	CPU limit (e.g. 1.0 = 1 vCPU)
--memory	Memory limit (e.g. 512m)
--memory-swap	Total memory + swap limit
--pids-limit	Maximum number of processes

9. SIGNALS, PID 1 AND CONTAINER EXITS

Topic	Key Points
Signals	Containers receive signals (SIGTERM, SIGKILL, etc.).
PID 1	First process in the container (must handle signals).
Graceful Shutdown	App should handle SIGTERM and exit cleanly.
Common Exit Codes	0 = success, 1 = general error, 137 = killed (OOM), 143 = terminated (SIGTERM).
Use an Init Process	Use tini or dumb-init to handle signals and zombies.

Check container exit code

```
docker ps -a
NAMES STATUS
web Exited (137)
api Exited (0)
```

View logs

```
docker logs <container>
docker logs -f <container>
```

10. CONNECTING CONTAINER SYMPTOMS TO THE HOST

Container Symptom	Check on the Host
Container is slow	High CPU, memory or I/O on host (top, htop, iostat, vmstat).
Container OOMKilled	Check host memory, dmesg, cgroup limits.
Cannot reach external network	Check host networking, DNS, iptables.
High disk usage	Check container writable layer and volumes (df, docker system df).
Container keeps restarting	Check logs, exit code, health checks, resource limits, underlying host issues.
Permission errors	Check volume mounts, user namespaces, host file permissions.

11. USEFUL COMMANDS

Inspect container <code>docker inspect <container></code> <code>docker top <container></code>	View resources <code>docker stats</code> <code>docker exec <container> top</code>
Check namespaces <code>lsns -p <container_pid></code> <code>readlink /proc/<pid>/ns/*</code>	Check cgroups <code>cat /sys/fs/cgroup/*/<container_id>/*</code>

12. KEY TAKEAWAYS

- Containers share the host kernel.
- Namespaces provide isolation.
- cgroups enforce resource limits.
- Images are built in layers.
- Use volumes for persistent data.
- Handle signals and PID 1 correctly.
- Always investigate container issues on the host.
- Understand the underlying Linux system.
- Set sensible resource limits.
- Keep images small and secure.

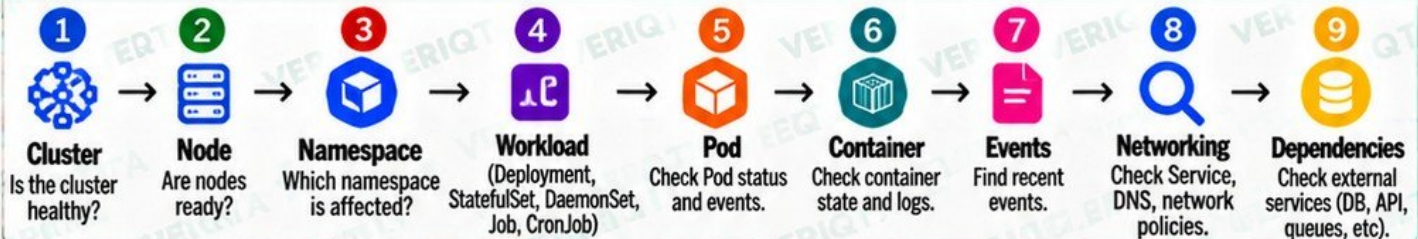
13. REMEMBER THIS

- A container is not a VM – it shares the host kernel.
- Most container problems are host problems.
- Always check logs, resource usage and host health.
- OOMKilled (exit 137) = memory issue.
- Exit 143 = SIGTERM (normal shutdown).
- Use proper resource limits.
- Keep images minimal.
- Understand how storage, networking and permissions work on the host.

Kubernetes Production Triage

A systematic investigation sequence from cluster to node, namespace, workload, Pod, container, events, logs, networking and dependencies. Use `kubectl` `get`, `describe`, `logs`, `top`, `events` and carefully controlled `exec` to quickly find the root cause.

1. KUBERNETES TRIAGE FLOW (BIG PICTURE)



2. CLUSTER AND NODE HEALTH

Command	Purpose
<code>kubectl cluster-info</code>	Check cluster control plane endpoints.
<code>kubectl get nodes -o wide</code>	View node status, roles and details.
<code>kubectl get componentstatuses</code>	Check control plane components (deprecated in newer versions).
<code>kubectl get --raw="/readyz"</code>	Check API server health.
<code>kubectl top nodes</code>	Check node CPU and memory usage (metrics-server).

3. NAMESPACE INVESTIGATION

Command	Purpose
<code>kubectl get ns</code>	List all namespaces.
<code>kubectl get all -n <ns></code>	View resources in a namespace.
<code>kubectl describe ns <ns></code>	Check namespace details, labels and events.
<code>kubectl get quota -n <ns></code>	Check resource quotas.
<code>kubectl get limitrange -n <ns></code>	Check default resource limits.

4. WORKLOAD INVESTIGATION

Command	Purpose
<code>kubectl get deploy -n <ns></code>	List Deployments.
<code>kubectl get sts -n <ns></code>	List StatefulSets.
<code>kubectl get ds -n <ns></code>	List DaemonSets.
<code>kubectl get job,cronjob -n <ns></code>	List Jobs and CronJobs.
<code>kubectl describe deploy <name> -n <ns></code>	Check desired state, replicas and events.
<code>kubectl rollout status deploy <name> -n <ns></code>	Check rollout progress.

5. POD AND CONTAINER INVESTIGATION

Command	Purpose
<code>kubectl get pods -n <ns></code>	List Pods and status.
<code>kubectl get pods -n <ns> -o wide</code>	View node, IP and more.
<code>kubectl describe pod <name> -n <ns></code>	Detailed info, events, conditions.
<code>kubectl get pod <name> -n <ns> -o yaml</code>	Full configuration (YAML).
<code>kubectl get containers -n <ns></code>	View containers in a Pod.

6. LOGS, EVENTS AND METRICS

Command	Purpose
<code>kubectl logs <pod> -n <ns></code>	View container logs.
<code>kubectl logs <pod> -c <container> -n <ns></code>	View logs for a specific container.
<code>kubectl logs <pod> --previous -n <ns></code>	View previous container logs (after restart).
<code>kubectl get events -n <ns> --sort-by=.lastTimestamp</code>	View recent events.
<code>kubectl top pod -n <ns></code>	Check Pod CPU/memory usage.

7. CONTROLLED EXEC USAGE

 Only use <code>exec</code> when necessary. Do not make changes on running containers unless required for investigation.	
Command	Purpose
<code>kubectl exec -it <pod> -n <ns> -- /bin/sh</code>	Open a shell (if available).
<code>kubectl exec -it <pod> -n <ns> -c <container> -- /bin/bash</code>	Exec into a specific container.
<code>kubectl exec <pod> -n <ns> -- <command></code>	Run a single command.

8. NETWORKING INVESTIGATION

Command	Purpose
<code>kubectl get svc -n <ns></code>	List Services.
<code>kubectl describe svc <name> -n <ns></code>	Check service details and endpoints.
<code>kubectl get endpoints -n <ns></code>	View service endpoints.
<code>kubectl get ingress -n <ns></code>	List Ingress resources.
<code>kubectl describe ingress <name> -n <ns></code>	Check Ingress configuration.
<code>kubectl get networkpolicy -n <ns></code>	View network policies.
<code>kubectl get pod -n <ns> -o wide</code>	Check Pod IPs and nodes.

9. COMMON POD STATES AND MEANINGS

Status	Meaning
Running	Pod is running normally.
Pending	Waiting for resources or scheduling.
CrashLoopBackOff	Container keeps crashing.
ImagePullBackOff	Cannot pull container image.
ErrImagePull	Image pull error (auth, network, tag).
OOMKilled	Container killed due to memory limit.
ContainerCreating	Container is being created.
Terminating	Pod is shutting down.
Completed	Job finished successfully.

10. DEPENDENCIES INVESTIGATION

Dependency	What to Check
Database	Connection, credentials, network policies, endpoint health.
External API	DNS, connectivity, TLS, latency, errors.
Message Queue	Broker availability, credentials, queue depth.
Cache (Redis)	Connectivity, memory, auth, latency.
Storage (S3, PVC)	Permissions, mount issues, capacity, latency.
Other Services	Use <code>curl</code> , <code>nc</code> , or temporary debug Pod to test connectivity.

11. REAL-WORLD TRIAGE EXAMPLE (STEP-BY-STEP)

1	Check cluster health	<code>kubectl get nodes, kubectl cluster-info</code>
2	Identify affected namespace	<code>kubectl get ns, kubectl get all -n <ns></code>
3	Check workload	<code>kubectl describe deploy <name> -n <ns></code>
4	Check Pod status	<code>kubectl get pods -n <ns>, kubectl describe pod <name> -n <ns></code>
5	Check events	<code>kubectl get events -n <ns> --sort-by=.lastTimestamp</code>
6	Check logs	<code>kubectl logs <pod> -n <ns></code>
7	Check networking	<code>kubectl get svc, endpoints, ingress -n <ns></code>
8	Check dependencies	Test DB, API, queue, storage connectivity
9	Use exec if needed	<code>kubectl exec -it <pod> -n <ns> -- /bin/sh</code> (read-only investigation)

12. KEY TAKEAWAYS

- Follow a systematic order: cluster → node → namespace → workload → Pod → container → events → logs → networking → dependencies.
- Use `kubectl describe` and `events` to understand what is happening.
- Check resource limits, OOM and node health.
- Verify networking, services and endpoints.
- Investigate dependencies outside the cluster.
- Use `exec` carefully and avoid making unnecessary changes.
- Always connect container symptoms back to the underlying Linux host.

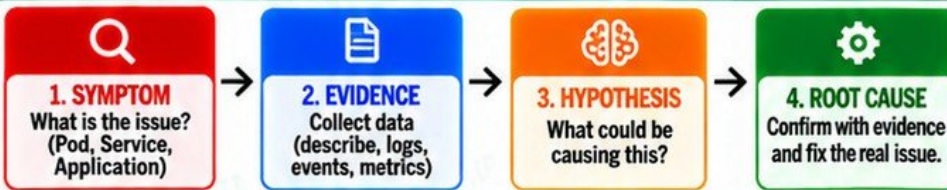
13. REMEMBER THIS

- Kubernetes does not hide problems, it surfaces them in `events`, logs and status.
- Most production issues are related to resources, networking, dependencies or misconfiguration.
- Check both the Kubernetes layer and the underlying node/host.
- One failing Pod can be caused by an external dependency.
- Never restart blindly. Understand the root cause first.

Kubernetes Failure Patterns

Learn how to investigate common Kubernetes failures using a consistent approach: symptom → evidence → hypothesis → root cause. Go beyond memorizing error names and understand what is really happening, where to look and how to fix it.

1. INVESTIGATION METHODOLOGY



USEFUL STARTING COMMANDS

```

kubectll get pods -A
kubectll describe pod <pod> -n <ns>
kubectll logs <pod> -n <ns> --previous
kubectll get events -A --sort-by=.lastTimestamp
kubectll get nodes
kubectll get svc -A
kubectll top pods -A
kubectll top nodes
  
```

2. COMMON KUBERNETES FAILURE PATTERNS

Failure Pattern	Typical Symptom	Where to Look (Evidence)	Common Root Causes
CrashLoopBackOff	Pod keeps restarting	- kubectll logs --previous - kubectll describe pod - kubectll get events	- Application crash - Misconfiguration (env, args, configs) - Dependency not ready - Resource limits (OOM, CPU)
ImagePullBackOff	Cannot pull container image	- kubectll describe pod - kubectll get events	- Wrong image name or tag - Registry authentication/permission - Network to registry - Image does not exist
Pending Pods	Pod stuck in Pending state	- kubectll describe pod - kubectll get events - kubectll get nodes	- Insufficient resources (CPU, memory, GPU) - Node selectors, taints, tolerations - PVC not bound - Scheduling constraints
Failed Probes	Liveness or readiness probe failing	- kubectll describe pod - kubectll logs - kubectll get events	- Application not ready - Wrong probe configuration (path, port, timeout) - Dependency not reachable - Slow startup
OOMKilled	Container terminated with OOMKilled	- kubectll describe pod - kubectll logs --previous - kubectll top pod	- Memory limit too low - Memory leak in application - High memory usage spikes - Insufficient node memory
Scheduling Failures	Pod cannot be scheduled	- kubectll describe pod - kubectll get events - kubectll get nodes	- Not enough resources - Node affinity/anti-affinity rules - Taints and tolerations - Required node labels not available
DNS Problems	Cannot resolve service or external DNS	- kubectll exec <pod> -- nslookup <name> - kubectll logs -n kube-system <dns-pod>	- CoreDNS not running or unhealthy - Wrong DNS configuration - Network policies blocking DNS - Upstream DNS issues
Volume Failures	Pod cannot mount volume	- kubectll describe pod - kubectll get pvc - kubectll get events	- PVC not bound - Storage class/driver issues - Permissions - Underlying storage system problems
Unavailable Services	Service has no endpoints or not reachable	- kubectll get svc -A - kubectll get endpoints <svc> -n <ns> - kubectll describe svc <svc> -n <ns>	- Pods not ready (failed probes) - Selector mismatch - Network policies - Backend pods crashing or not running

3. DETAILED EXAMPLES

A. CrashLoopBackOff

```

$ kubectll describe pod web-abc -n app
Events:
  Back-off restarting failed container
  
```

- Hypothesis:** Application crash or bad config.
- Check:** logs, env vars, config files, dependencies.

B. ImagePullBackOff

```

$ kubectll describe pod api-xyz -n app
Events:
  Failed to pull image "myapp:latest"
  Error: ImagePullBackOff
  
```

- Hypothesis:** Wrong image, auth or network.
- Check:** image name, registry creds, network access.

C. OOMKilled

```

$ kubectll describe pod worker -n app
Last State: Terminated
Reason: OOMKilled
  
```

- Hypothesis:** Container exceeded memory limit.
- Check:** resource limits, memory usage, app logs.

D. Pending Pod (Insufficient Resources)

```

$ kubectll describe pod db-0 -n app
Events:
  0/3 nodes are available: insufficient cpu.
  
```

- Hypothesis:** Not enough cluster resources.
- Check:** node capacity, requests/limits, taints.

4. SYMPTOM → EVIDENCE → HYPOTHESIS → ROOT CAUSE (EXAMPLE)

Symptom	Evidence (Commands/Output)	Hypothesis	Root Cause
Pod stuck Pending	kubectll describe pod Events: insufficient memory	Node lacks resources.	Increase node size or adjust requests/limits.
Readiness probe failing	kubectll describe pod Readiness probe failed: connection refused	App not ready or wrong probe config.	Fix app startup or probe settings.
Service unavailable	kubectll get endpoints No endpoints	Pods not ready or selector mismatch.	Fix pod issues or correct selector.
DNS resolution failing	kubectll exec <pod> -- nslookup service ;; connection timed out	CoreDNS or network issue.	Restart CoreDNS or check network policies.

5. KEY TAKEAWAYS

- Always follow symptom → evidence → hypothesis → root cause.
- Use kubectll describe, logs, events and metrics.
- Check both application and cluster level issues.
- Look at node resources, networking, DNS and storage.
- Do not rely on error names alone.
- Validate the fix and monitor for recurrence.

6. REMEMBER THIS

- A Kubernetes error is a symptom, not always the root cause.
- Check multiple layers: application, cluster, node, network and dependencies.
- Use real evidence before making changes.
- Fix the cause, not just the symptom.
- Document what you find to help future investigations.



@veriqta_academy



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Observability for Infrastructure Engineers

Understand metrics, logs, traces, events, dashboards, and alerts. Learn the difference between signals and symptoms. Master the golden signals: latency, traffic, errors, and saturation. See how to correlate multiple signals to find the real root cause during an incident.

1. WHAT IS OBSERVABILITY?

					
Metrics	Logs	Traces	Events	Dashboards	Alerts
Numeric data over time (what is happening)	Detailed records of what happened (why it happened)	Request flow across services (where it happened)	State changes in the system (what changed)	Visualize and combine signals (see the big picture)	Notify on abnormal conditions (take action early)

2. SIGNALS vs SYMPTOMS

Signal	Symptom
✓ Measurable evidence from the system.	✗ What you observe on the surface.
✓ Tells you what is actually happening.	✗ Does not always show the real cause.
✓ Examples: high CPU, 5xx rate, increased latency, error logs, queue depth.	✗ Examples: site is slow, users cannot login, pods keep restarting.

3. THE GOLDEN SIGNALS (FOUR KEY METRICS)

Latency	Traffic	Errors	Saturation
			
- How long requests take. - High latency means the system is slow. - Examples: p50, p95, p99 response time.	- Amount of requests coming in. - Sudden spikes or drops can indicate issues. - Examples: requests per second (RPS).	- Rate of failed requests. - High error rate indicates failures. - Examples: HTTP 5xx, 4xx, application errors.	- How full the system is (resources). - When resources are exhausted, performance degrades. - Examples: CPU, memory, disk, network, queue length.

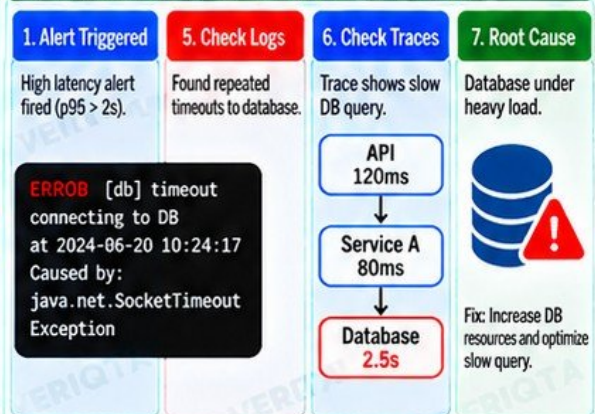
4. OBSERVABILITY STACK (EXAMPLE)

Metrics	→ 	Prometheus (or cloud monitoring)	Collection, storage and alerting.
Logs	→ 	Grafana Loki (or ELK, OpenSearch)	Log aggregation and search.
Traces	→ 	OpenTelemetry (or Jaeger, Tempo)	Distributed tracing across services.
Events	→ 	Kubernetes Events (Events API)	Cluster and application events.
Dashboards	→ 	Grafana (or cloud dashboards)	Visualize metrics, logs and traces.
Alerts	→ 	Alertmanager (or cloud alerting)	Send alerts to Slack, Email, PagerDuty, etc.

5. CORRELATING MULTIPLE SIGNALS DURING AN INCIDENT



6. EXAMPLE: INCIDENT INVESTIGATION FLOW



7. EXEFCUL COMMANDS AND QUERIES (EXAMPLES)

1. Alert Triggered High latency alert fired (p95 > 2s). Latency (p95)	2. Check Traffic Traffic is normal (no major spike). Requests per Second	3. Check Errors 5xx error rate increased. HTTP 5xx Rate	4. Check Saturation CPU on one node is high. Node CPU Usage	5. Check Logs Found repeated timeouts to database. ERROR [db] timeout connecting to DB at 2024-06-20 10:24:17 Caused by: java.net.SocketTimeoutException	6. Check Traces Trace shows slow DB query. API 120ms Service A 80ms Database 2.5s
--	---	--	--	--	--

7. USEFUL COMMANDS AND QUERIES (EXAMPLES)

Tool / Area	Command / Query	Purpose
Metrics (Prometheus)	rate(http_requests_total[5m])	View request rate.
Metrics (Prometheus)	histogram_quantile(0.95, rate(http_request_duration[5m]))	Check p95 latency.
Logs (Loki)	[app="api"] = "error"	Search error logs.
Traces (Jaeger)	Trace ID: 4f3e2a7c9d8b1	View request flow.
Kubernetes Events	kubectl get events --sort-by=.lastTimestamp	View recent events.
Kubernetes Logs	kubectl logs <pod> -n <ns> --tail=100	Check container logs.
Kubernetes Top	kubectl top pods -n <ns>	Check resource usage.

8. KEY TAKEAWAYS

- ✓ Use metrics, logs, traces and events together.
- ✓ Signals show what is happening, symptoms show what you notice.
- ✓ Focus on the golden signals: latency, traffic, errors and saturation.
- ✓ Correlate multiple signals to find the real root cause.
- ✓ Build useful dashboards and meaningful alerts.
- ✓ Investigate methodically: user impact → metrics → logs → traces → events → root cause.
- ✓ Don't rely on a single signal.
- ✓ Use observability to reduce MTTR and prevent recurrence.

9. REMEMBER THIS

- High latency is a signal.
- 5xx errors are a signal.
- A restarting pod is a symptom.
- Always look at multiple signals.
- Dashboards should answer real questions.
- Alerts should be actionable and not noisy.
- Observability helps you find the cause, not just the symptom.



@veriqta_academy



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Capacity, Scaling and Performance

Learn how to plan, measure and improve system capacity and performance. Understand requests vs limits, utilization, headroom, horizontal and vertical scaling, autoscaling, bottlenecks, saturation, load testing, forecasting and capacity planning. Answer the senior question: **"What breaks first if traffic doubles?"**

1. KEY CONCEPTS

						
Requests Minimum resources guaranteed for a container or service.	Limits Maximum resources a container can use.	Utilization How much of the available resources are being used.	Headroom Spare capacity to handle traffic spikes and failures.	Scaling Increase or decrease resources to match demand.	Bottlenecks The component that limits overall system performance.	Saturation When resources are fully used and performance degrades.

2. REQUESTS vs LIMITS (KUBERNETES EXAMPLE)

Field	Meaning
requests.cpu	Minimum CPU reserved.
requests.memory	Minimum memory reserved.
limits.cpu	Maximum CPU allowed.
limits.memory	Maximum memory allowed.

```

apiVersion: v1
kind: Pod
metadata:
  name: web
spec:
  containers:
    - name: web
      image: nginx
      resources:
        requests:
          cpu: "250m"
          memory: "512Mi"
        limits:
          cpu: "500m"
          memory: "1Gi"

```

3. SCALING STRATEGIES

Vertical Scaling (Scale Up) 	Horizontal Scaling (Scale Out) 	Autoscaling 
- Increase CPU, memory or storage on existing nodes or instances. - Simple but has limits. - Useful for stateful workloads.	- Increase the number of replicas or nodes. - Handles higher load and improves availability. - Used with load balancers and autoscaling.	- Automatically adjusts the number of replicas or nodes based on metrics (e.g. CPU, memory, custom metrics). - Helps handle traffic spikes and reduces cost.

4. TYPES OF BOTTLENEKS

Bottleneck Type	Examples
CPU	High CPU usage, CPU throttling, slow processing.
Memory	OOMKilled, swapping, high memory usage.
Disk I/O	High read/write latency, slow storage.
Network	High latency, packet loss, low bandwidth.
Application	Inefficient code, database queries, external API latency.
Database	Slow queries, connection limits, lock contention.
External Services	Third-party API slowness, rate limits, outages.

5. MEASURING PERFORMANCE

Metric	What to Measure
CPU	Usage, throttling, load average.
Memory	Usage, limits, OOM events.
Disk	Read/write throughput, IOPS, latency.
Network	Requests per second, latency, error rate.
Application	Response time, throughput, error rate.
Database	Query performance, connection count, slow queries.
Business	Active users, transactions, revenue impact.

6. LOAD TESTING (EXAMPLE)

```

# Using k6 (example)
k6 run load-test.js

# Simple HTTP load test
import http from 'k6/http';
import { sleep } from 'k6';
export default function () {
  http.get('https://your-app.com');
  sleep(1);
}

```

What Load Testing Helps You Find

- System behavior under real traffic.
- Breaking points and bottlenecks.
- Latency and error rate at scale.
- Database and external dependency limits.
- Capacity requirements for future growth.

7. FORECASTING AND CAPACITY PLANNING

- Collect historical usage data (CPU, memory, requests, storage).
- Identify growth trends (daily, weekly, seasonal).
- Predict future demand (e.g. 3, 6, 12 months).
- Plan capacity with headroom (account for spikes and failures).
- Review and adjust regularly (business growth, new features).

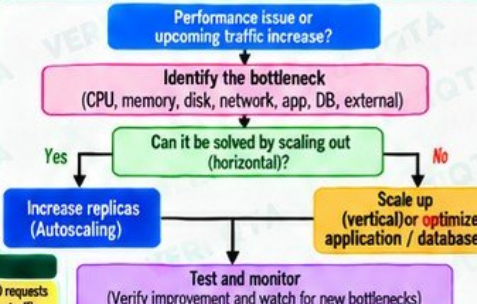
Common Methods

- Trend analysis
- Seasonal forecasting
- Growth rate estimation
- Capacity modeling
- Regular review and adjustment

8. WHAT BREAKS FIRST IF TRAFFIC DOUBLES?

1 Load Balancer	Can it handle 2X connections and throughput?
2 Network	Bandwidth and latency limits.
3 Application Pods	CPU, memory, and replica count.
4 Database	Connection limits and query performance.
5 External APIs	Rate limits and slower responses.
6 Storage	Read/write IOPS and latency.
7 Other Dependencies	DNS, message queues, caches, etc.

9. EXAMPLE: SCALING DECISION FLOW



10. KEY TAKEAWAYS

- Understand requests, limits, utilization and headroom.
- Use horizontal and vertical scaling based on the bottleneck.
- Configure autoscaling for dynamic demand.
- Identify and remove bottlenecks.
- Use load testing to find limits.
- Plan capacity based on real data and growth trends.
- Always consider: **What breaks first if traffic doubles?**
- Continuously monitor and review capacity.

11. USEFUL COMMANDS AND QUERIES

Command	Purpose
kubectl top nodes	Check node CPU and memory usage.
kubectl top pods -n <ns>	Check pod resource usage.
kubectl describe hpa -n <ns>	View Horizontal Pod Autoscaler details.
kubectl get hpa -n <ns>	List autoscalers.
kubectl get nodes	Check node capacity and allocatable resources.

12. REAL-WORLD SCENARIO

- Your application normally handles 1,000 requests per second. After a marketing campaign, traffic suddenly doubles to 2,000 requests per second.
- Which component is most likely to break first?
 - What metrics will you check?
 - How will you scale or optimize?
 - What is your plan to prevent this in the future?

13. REMEMBER THIS

- Scaling is not just about adding more resources.
- Find the real bottleneck.
- Use metrics, logs and traces to guide decisions.
- Plan for future growth, not just today.
- Always test, monitor and validate changes.
- Answer like a senior engineer: "What breaks first if traffic doubles?"


[@veriqta_academy](#)

[@veriqta](#)

[@veriqta](#)

[@veriqta](#)

[@veriqta](#)

[@veriqta](#)

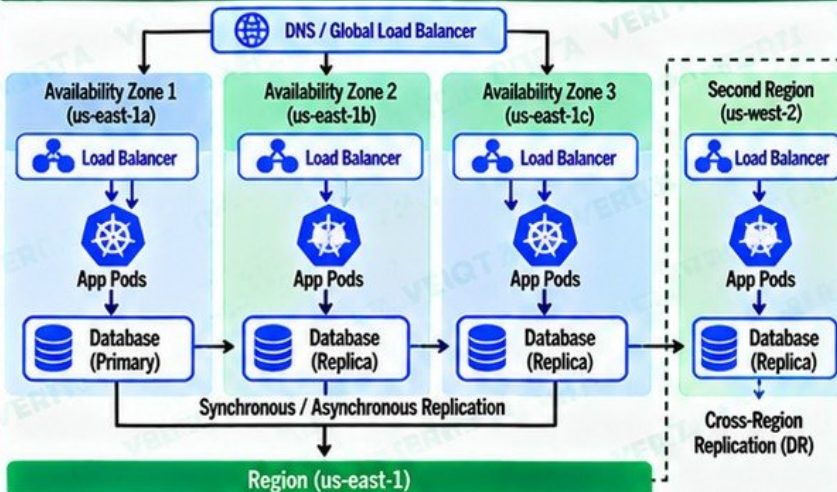
High Availability and Failure Domains

Learn how to design and operate highly available systems. Understand redundancy, zones, regions, quorum, replication, failover, single points of failure, graceful degradation, RTO and RPO. Learn why duplicated infrastructure does not automatically equal high availability.

1. KEY CONCEPTS



3. EXAMPLE: HIGHLY AVAILABLE ARCHITECTURE



6. COMMON SINGLE POINTS OF FAILURE (SPOF)

Component	Examples
Compute	Single node, single VM, single Kubernetes node.
Storage	Single database instance, single volume.
Network	Single load balancer, single firewall, single AZ.
DNS	Only one DNS provider or region.
Identity	Single IAM system or directory.
External Services	Third-party APIs with no fallback.
Monitoring	No visibility during an outage.
Human Process	Only one person with access or knowledge.

7. REPLICATION AND QUORUM

Concept	Description
Replication	Keep multiple copies of data across nodes, zones or regions.
Quorum	Minimum number of nodes required to make a decision (e.g. 3 of 5).
Leader / Follower	One node handles writes, others replicate (e.g. database).
Consensus	Ensures data remains consistent even during failures (e.g. etcd, Raft, ZooKeeper).

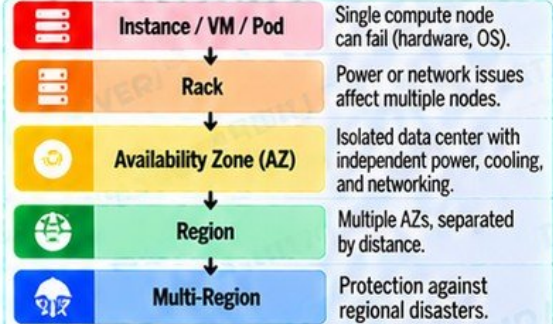
9. FAILURE SCENARIOS AND IMPACT

Scenario	Potential Impact
Single Pod Failure	Minimal impact (handled by replication).
Node Failure	Pods rescheduled to healthy nodes.
AZ Failure	Traffic shifts to other AZs in the region.
Region Failure	Failover to secondary region (DR).
Database Primary Failure	Failover to replica, small downtime.
Network Partition	Service isolation, split brain risk.
Third-Party API Outage	Feature degradation or fallback.

10. DESIGN CONSIDERATIONS

- ✓ Distribute resources across zones and regions.
- ✓ Eliminate single points of failure.
- ✓ Use automated health checks and failover.
- ✓ Define and test RTO and RPO.
- ✓ Design for graceful degradation.
- ✓ Validate backups and restore procedures.
- ✓ Monitor key dependencies.
- ✓ Regularly test disaster recovery scenarios.

2. FAILURE DOMAINS (HIERARCHY)



4. RTO AND RPO

Term	Meaning
RTO (Recovery Time Objective)	Maximum acceptable time to restore the service after a failure.
RPO (Recovery Point Objective)	Maximum acceptable amount of data that can be lost.
Example	RTO: 15 minutes RPO: 5 minutes (Choose values based on business requirements).

5. DUPLICATION ≠ HIGH AVAILABILITY

Just Duplicated Infrastructure	True High Availability
✗ Two servers in the same zone.	✓ Resources distributed across zones or regions.
✗ Two databases with no failover.	✓ Automated failover and health checks.
✗ No automated recovery.	✓ Independent dependencies (network, power, DNS).
✗ Shared dependencies (e.g. single network, single DNS).	✓ Tested recovery procedures.
✗ Fails during zone or network outage.	✓ Meets defined RTO and RPO.

8. FAILOVER STRATEGIES

Strategy	Description
Automatic Failover	System detects failure and switches traffic automatically (health checks, routing).
Manual Failover	Engineer initiates failover.
Active-Passive	One active, one standby resource.
Active-Active	Resources serve traffic simultaneously (requires conflict handling).
Blue-Green	Switch to a healthy environment with minimal downtime.
Canary / Progressive	Gradually shift traffic to new resources.

11. KEY TAKEAWAYS

- ✓ High availability is about design, not just duplication. Understand and map your failure domains.
- ✓ Remove single points of failure.
- ✓ Use replication, quorum and automated failover.
- ✓ Define and meet RTO and RPO.
- ✓ Design for graceful degradation, not complete outage.
- ✓ Test your architecture regularly.
- ✓ Know what breaks first if a zone, region or key component fails.

Production Change and Rollback Strategy

Learn how to plan, execute and safely manage changes in production. Understand change risk, blast radius, canary deployment, blue-green deployment, feature flags, maintenance windows, database compatibility, rollback vs roll-forward, verification and stop conditions.

! Before changing production, know how you will detect failure and recover.

1. CHANGE RISK AND BLAST RADIUS

Change Risk



- Size of the change.
- Complexity.
- Number of components affected.
- Likelihood of failure.
- Ease of rollback.

Blast Radius



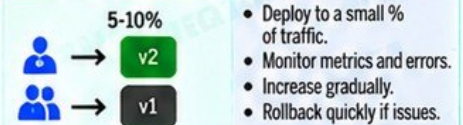
- Which services are affected?
- Which environments (AZ/region)?
- Which customers or users?
- What data could be impacted?
- How fast can it spread?

2. CHANGE LIFECYCLE



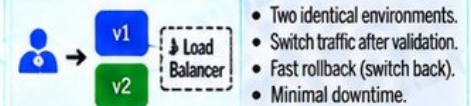
3. DEPLOYMENT STRATEGIES

Canary Deployment



- Deploy to a small % of traffic.
- Monitor metrics and errors.
- Increase gradually.
- Rollback quickly if issues.

Blue-Green Deployment



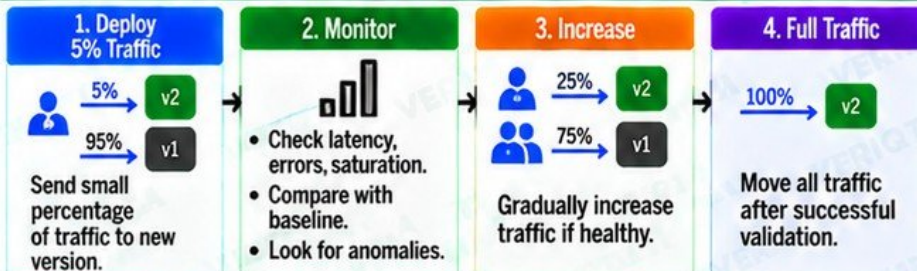
- Two identical environments.
- Switch traffic after validation.
- Fast rollback (switch back).
- Minimal downtime.

Feature Flags

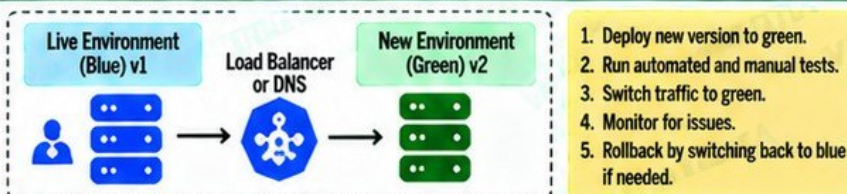


- Deploy code without enabling the feature.
- Enable for specific users.
- Gradual rollout.
- Quickly disable if problems.

4. EXAMPLE: CANARY DEPLOYMENT FLOW



5. EXAMPLE: BLUE-GREEN DEPLOYMENT FLOW



1. Deploy new version to green.
2. Run automated and manual tests.
3. Switch traffic to green.
4. Monitor for issues.
5. Rollback by switching back to blue if needed.

6. DATABASE CHANGES AND COMPATIBILITY



- Use backward-compatible changes (expand and contract).
- Avoid breaking schema changes.
- Test with realistic data and application versions.
- Consider migration time and locking.
- Verify application and database version compatibility.
- Plan for rollback or roll-forward (some changes cannot be easily rolled back).

7. ROLLBACK vs ROLL-FORWARD

Rollback

- Revert to previous version.
- Useful for quick recovery.
- May not be possible for database or schema changes.
- Requires keeping previous version and configuration.

Roll-Forward

- Fix the issue and deploy a new version.
- Used when rollback is not possible or not safe (e.g., database changes).
- Requires a tested fix.
- Often preferred for complex environments.

8. MAINTENANCE WINDOWS



- Schedule changes during low-traffic periods.
- Communicate with stakeholders.
- Have on-call and support ready.
- Prepare rollback or roll-forward plan.
- Monitor closely during and after the change.

9. VERIFICATION AFTER CHANGE



- Check application health (logs, metrics, traces).
- Verify key user workflows.
- Compare metrics with baseline (latency, error rate, traffic).
- Check infrastructure and dependencies.
- Confirm database state and data integrity.
- Ensure no unexpected side effects.

10. STOP CONDITIONS (ABORT THE CHANGE)



- Error rate increases beyond threshold.
- Latency is significantly higher than baseline.
- Critical user workflows are failing.
- Resource usage (CPU, memory, etc.) is abnormal.
- Unexpected errors in logs or monitoring.
- Any sign of data corruption or loss.
- SLOs or business impact is at risk.

11. CHANGE CHECKLIST

- ✓ Understand the change and its risk.
- ✓ Identify blast radius.
- ✓ Have a tested rollback or roll-forward plan.
- ✓ Test in non-production.
- ✓ Use canary, blue-green or feature flags where possible.
- ✓ Schedule a maintenance window (if needed).
- ✓ Monitor and verify after deployment.
- ✓ Be ready to stop if issues are detected.

12. KEY TAKEAWAYS



- Plan, test and validate before changing production.
- Use deployment strategies to reduce risk.
- Understand the blast radius.
- Not all changes can be rolled back — sometimes you must roll forward.
- Database changes require special care.
- Always have stop conditions and monitoring in place.
- Duplicated infrastructure does not automatically equal high availability.
- Before changing production, know how you will detect failure and recover.



@veriqta_academy



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Incident Response and Root Cause Analysis

Learn how to respond to incidents using a structured process: detection → triage → mitigation → stabilization → investigation → recovery → review. Understand severity, incident command, timelines, communication, evidence preservation, contributing factors, root cause, and corrective actions. Always focus on restoring service before proving theories.

! Restore service first. You can investigate and find the root cause after the system is stable.

1. INCIDENT RESPONSE LIFECYCLE



2. INCIDENT SEVERITY LEVELS (EXAMPLE)

Severity	Impact	Examples	Response Time
SEV-1	Major outage, critical service, down, large user impact.	Production down, data loss, security breach.	Immediate (24/7)
SEV-2	Significant degradation, core features affected.	High error rate, partial outage.	Within 30 minutes
SEV-3	Minor impact, non-critical service.	Feature not working, increased latency.	Within 2-4 hours
SEV-4	Low impact, cosmetic or informational.	UI issue, small bug, no user impact.	Within 1 business day

3. INCIDENT COMMAND AND ROLES

Role	Responsibilities
Incident Commander (IC)	Leads the response, makes decisions, maintains focus.
Technical Lead	Coordinates technical investigation and fixes.
Communications Lead	Handles internal and external communication.
Scribe	Maintains timeline, decisions and actions.
Subject Matter Experts	Provide technical support (DB, network, security, etc).
Stakeholders	Informed regularly based on severity.

4. INCIDENT TIMELINE (EXAMPLE)

Time (UTC)	Event	Details
10:00	Alert Triggered	High error rate detected on API.
10:02	Triage	Confirmed user impact, severity SEV-1.
10:05	Mitigation	Scaled up replicas and restarted pods.
10:15	Stabilization	Error rate reduced, service recovering.
10:30	Investigation	Analyzing logs, metrics and recent changes.
11:20	Recovery	Full service restored.
12:00	Review	Document findings and create action items.

5. COMMUNICATION DURING AN INCIDENT

Audience	What to Share	Frequency
Internal Team	Current status, impact, actions, next steps.	Regular updates (e.g. every 15-30 min)
Leadership	Business impact, ETA, key decisions.	Regular updates (e.g. every 30-60 min)
Customers / Users	Acknowledgement, impact, progress, resolution.	Based on severity (e.g. every 30-60 min)

6. EVIDENCE PRESERVATION

- ✓ Collect and preserve logs (app, system, network).
- ✓ Capture metrics and dashboards (screenshots if needed).
- ✓ Save configuration files and recent changes.
- ✓ Document commands executed and their output.
- ✓ Preserve timestamps and timeline.
- ✓ Do not modify or delete evidence until investigation is complete.

7. ROOT CAUSE ANALYSIS PROCESS

- 1 Identify the problem → What happened?
- 2 Gather evidence → Logs, metrics, configs, changes.
- 3 Find contributing factors → What made it worse?
- 4 Determine root cause → The primary reason.
- 5 Validate the root cause → Reproduce if possible.
- 6 Define corrective actions → Prevent recurrence.

8. EXAMPLE: FROM SYMPTOM TO ROOT CAUSE

Stage	Example (API Outage)
Symptom	High 5xx errors, users cannot log in.
Evidence	Logs show database connection timeouts.
Contributing Factor	Connection pool exhausted due to traffic spike.
Root Cause	Recent deployment reduced DB connection limit.
Fix	Increase connection pool and redeploy.
Corrective Action	Add capacity checks in deployment pipeline.

9. COMMON INCIDENT PITFALLS

- ✗ Spending too much time proving theories instead of restoring service.
- ✗ Poor communication or delayed updates.
- ✗ Not capturing evidence.
- ✗ Making untested changes in production.
- ✗ Ignoring contributing factors.
- ✗ No follow-up or preventive actions after resolution.

10. INCIDENT REVIEW AND ACTION ITEMS

- ✓ Write a clear incident summary.
- ✓ Include timeline, impact and root cause.
- ✓ Identify what went well and what went wrong.
- ✓ Define and track corrective and preventive actions.
- ✓ Update runbooks, monitoring and alerts.
- ✓ Share learnings with the team.

11. KEY TAKEAWAYS

- ✓ Focus on restoring service before proving theories.
- ✓ Use a structured incident response process.
- ✓ Clear communication reduces confusion and builds trust.
- ✓ Preserve evidence for accurate investigation.
- ✓ Find the root cause and address contributing factors.
- ✓ Implement corrective actions to prevent recurrence.
- ✓ Every incident is a learning opportunity.

Senior Infrastructure Engineer's Production Checklist

A final field-reference checklist for operating, troubleshooting and validating production systems. Use this to quickly assess health, security, observability, capacity, changess, blast radius, rollback, backups, dependencies and recovery. Never assume the change worked. Verify the system state.

1. IS IT HEALTHY?



- ✓ Are services running?
- ✓ Are pods/nodes/instances healthy?
- ✓ Are there errors or restarts?
- ✓ Are key transactions working?
- ✓ Are SLIs/SLOs within target?

Quick Checks (Examples)

```
systemctl status <service>
kubectl get pods -A
kubectl get nodes
curl -I https://your-app
kubectl top nodes,pods
```

2. IS IT SECURE?



- ✓ Are access controls correct?
- ✓ Are secrets protected?
- ✓ Are there unauthorized changes?
- ✓ Are patches up to date?
- ✓ Any suspicious activity?

Quick Checks (Examples)

```
kubectl get roles,rolebindings -A
aws iam get-user
sudo last 10
ss -tulpen
journalctl -u ssh -n 50
```

3. IS IT OBSERVABLE?



- ✓ Are metrics, logs and traces flowing?
- ✓ Are dashboards showing real data?
- ✓ Are alerts working?
- ✓ Can you investigate quickly?
- ✓ Are key events being captured?

Quick Checks

```
curl <metrics-endpoint>
kubectl logs <pod> --tail=100
kubectl get events -A
kubectl describe <resource>
Check Grafana / Prometheus / Loki
```

4. IS CAPACITY SUFFICIENT?



- ✓ Are CPU, memory, disk and network within limits?
- ✓ Is there enough headroom?
- ✓ Are we close to saturation?
- ✓ Will it handle traffic spikes?
- ✓ Is autoscaling working?

Quick Checks

```
kubectl top nodes,pods
df -h && free -h
iostat -xz 1
kubectl get hpa
Check cloud monitoring dashboards
```

5. WHAT CHANGED?



- ✓ What was deployed or modified?
- ✓ Who made the change?
- ✓ When did it happen?
- ✓ Was it infrastructure, code or config?
- ✓ Is the change fully rolled out?

Quick Checks

```
kubectl rollout history <deploy>
git log --online
kubectl describe <resource>
Check CI/CD pipeline logs
Review change ticket / timeline
```

6. WHAT IS THE BLAST RADIUS?



- ✓ Which services are affected?
- ✓ Which environments (AZ/region)?
- ✓ Which users or customers?
- ✓ Are dependencies also impacted?
- ✓ Can the issue spread?

Quick Checks

```
kubectl get endpoints
kubectl get svc -A
traceroute <service>
Check load balancer and DNS
Review architecture diagram
```

7. CAN WE ROLL BACK?



- ✓ Is a previous version available?
- ✓ Is the rollback process tested?
- ✓ Will rollback cause other issues?
- ✓ Is data backward compatible?
- ✓ How long will rollback take?

Quick Checks

```
kubectl rollout undo <deploy>
helm rollback <release> <rev>
Check previous AMI/image tag
Verify database migration plan
Review feature flag status
```

8. ARE BACKUPS USABLE?



- ✓ Are backups recent and complete?
- ✓ Have restores been tested?
- ✓ Is the RPO acceptable?
- ✓ Are backups in a different region?
- ✓ Can we restore quickly?

Quick Checks

```
aws s3 ls s3://<backup-bucket>
Check snapshot status
Test restore in non-prod
Verify backup retention policy
Check RTO/RPO documentation
```

9. ARE DEPENDENCIES HEALTHY?



- ✓ Are databases reachable?
- ✓ Are external APIs responding?
- ✓ Are message queues healthy?
- ✓ Are DNS and load balancers working?
- ✓ Are third-party services available?

Quick Checks

```
kubectl get endpoints -A
nslookup <service>
curl -I <external-api>
kubectl get pods (db, mq, etc)
Check cloud provider status page
```

10. WHAT EVIDENCE CONFIRMS RECOVERY?



- ✓ Are error rates back to normal?
- ✓ Are key transactions successful?
- ✓ Are metrics stable for a period?
- ✓ Are users confirming it works?
- ✓ Is there no hidden impact elsewhere?

Quick Checks

```
Compare metrics before/after
kubectl get events -A
Check logs for errors
Run synthetic tests
Get user confirmation
```

11. END-TO-END PRODUCTION CHECKLIST

Area	Key Questions
Host	Is the host healthy (CPU, memory, disk, services)?
Network	Is networking working (DNS, connectivity, latency)?
Security	Are access controls, patches and secrets correct?
Traffic	Are requests flowing and error rates normal?
Infrastructure Automation	Did the automation run successfully?
Cloud Identity	Are users, roles and permissions correct?
Containers	Are containers running and healthy?
Kubernetes	Are pods, nodes and cluster components healthy?
Observability	Are metrics, logs, traces and alerts working?
Scale	Is capacity sufficient and autoscaling working?
Resilience	Are we protected against failures (multi-AZ/region)?
Production Change	Was the change successful and verified?
Incidents	Is the incident resolved and lessons captured?

12. SENIOR PRINCIPLES



- ✓ Restore service before proving theories.
- ✓ Work from evidence, not assumptions.
- ✓ Validate the full system state after any change.
- ✓ Always consider the blast radius.
- ✓ Have a rollback and recovery plan.
- ✓ Use observability to confirm, not assume.
- ✓ Document and learn from every incident.

13. FINAL VERIFICATION STEPS



- ✓ Run functional and synthetic tests.
- ✓ Check key metrics for stability.
- ✓ Verify no hidden errors in logs.
- ✓ Confirm with real users if possible.
- ✓ Monitor for a sustained period.
- ✓ Close the incident/change only when fully confident.

14. REMEMBER THIS



- ✓ Duplicated infrastructure ≠ high availability.
- ✓ No change is complete until verified.
- ✓ Backups are only useful if they can be restored.
- ✓ A healthy system is observable, secure and scalable.
- ✓ Small issues can become major incidents.
- ✓ Good documentation saves time under pressure.
- ✓ Continuous verification prevents repeat failures.



Never assume the change worked. Verify the system state.



@veriqta_academy



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta